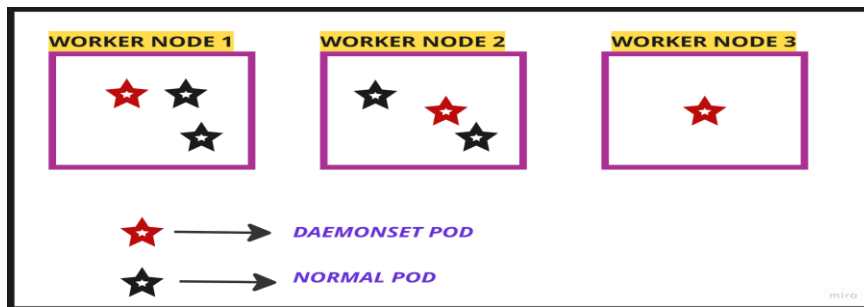


DAEMONSET

- A DaemonSet ensures that a Pod runs on every Worker Node in the Kubernetes cluster.
- When a new Worker Node is added, Kubernetes automatically creates a Pod on that node.
- Commonly used for monitoring, logging, and networking agents.



STATEFULSET

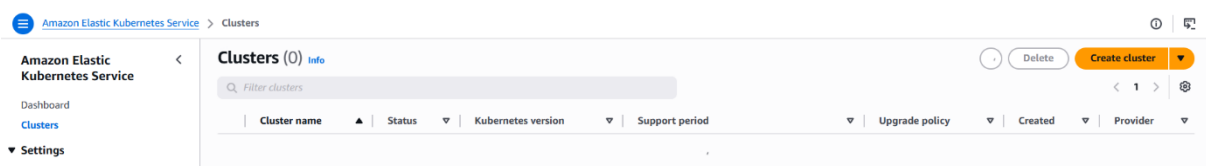
- A StatefulSet is used for applications that require stable identities and persistent storage.
- Each Pod gets a unique name (for example: mysql-stateful-0, mysql-stateful-1).
- Pods are created and deleted in an ordered sequence (0, 1, 2, ...).
- Uses a ClusterIP (Headless) Service for stable network identities.
- Commonly used for databases such as MySQL, MongoDB, and PostgreSQL.
- Pods are terminated in reverse order.
- Each Pod has its own persistent storage (PVC).

HANDS-ON FOR DAEMONSET AND STATEFULSET

DAEMONSET

Screenshot: Open Amazon EKS

- Open the **AWS Management Console**.
- Search for **EKS**.
- Select **Elastic Kubernetes Service (EKS)**.
- Click **Create Cluster**.



Screenshot: Configure Cluster Name

- Select **Custom Configuration**.
- Enter the **Cluster Name**.

Amazon Elastic Kubernetes Service > Create EKS cluster

Configure cluster

Step 2
Specify networking
Step 3
Configure observability
Step 4
Select add-ons
Step 5
Configure selected add-ons settings
Step 6
Review and create

Configuration options Info

Choose how you would like to configure the cluster.

☐ Quick configuration (with EKS Auto Mode)
Quickly create a cluster with production-grade default settings. The configuration uses EKS Auto Mode to automate infrastructure tasks like creating nodes and provisioning storage.

☒ Custom configuration
To change default settings prior to creation, choose this option. This configuration gives the option to use EKS Auto Mode and customize the cluster's configuration.

EKS Auto Mode Info

Choose if you would like to use EKS's Auto Mode.

☒ Use EKS Auto Mode
EKS automates routine cluster tasks for compute, storage, and networking. When a new pod can't fit onto existing nodes, EKS creates a new node. EKS combines cluster infrastructure managed by AWS with integrated Kubernetes capabilities to meet application compute needs. [View pricing](#)

► Included capabilities

Cluster configuration Info

Name
Enter a unique name for this cluster. This property cannot be changed after the cluster is created.

myclustet

The cluster name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 100.

Screenshot: Select Cluster IAM Role

- Click **Create New Role**.
- Create a new IAM role.
- Select the **created IAM role** from the **Cluster IAM Role** dropdown.
- Click **Next** to continue.

Cluster IAM role

To access other AWS services and resources your Kubernetes control plane needs an IAM role. Choose a role with the right permissions for your Kubernetes control plane.

AmazonEKSClusterRole

Create new role

Q

EKS Kubernetes control plane service-assumable roles

AmazonEKSClusterRole
arn:aws:iam::301804611986:role/AmazonEKSClusterRole ✓

AWSServiceRoleForAmazonEKS
arn:aws:iam::301804611986:role/aws-service-role/eks.amazonaws.com/AWSServiceRoleForAmazonEKS

Screenshot: Configure Cluster Access

- Select **EKS API and ConfigMap**.
- Click **Next**.

Cluster authentication mode Info

Configure which source the cluster will use for authenticated IAM principals.

☐ EKS API
The cluster will source authenticated IAM principals only from EKS access entry APIs.

☒ EKS API and ConfigMap
The cluster will source authenticated IAM principals from both EKS access entry APIs and the aws-auth ConfigMap.

Screenshot: Configure Networking

- Select the **Security Group**.

- Select **Public** access.
- Click **Create**.
- EKS cluster creation started successfully.

Additional security groups [Info](#)

EKS automatically creates a cluster security group on cluster creation to facilitate communication between worker nodes and control plane. Optionally, choose additional security groups to apply to the EKS-managed Elastic Network Interfaces that are created in your control plane subnets. To create a new security group, go to the corresponding page in the [VPC console](#).

Select security groups

☒ sg-01fd0fe3f335a0753 | jai
☐ sg-0230b4c40073e70b6 | launch-wizard-20
launch-wizard-20 created 2026-05-25T10:50:38.906Z

Cluster endpoint access [Info](#)

Configure access to the Kubernetes API server endpoint.

- ☒ **Public**
The cluster endpoint is accessible from outside of your VPC. Worker node traffic will leave your VPC to connect to the endpoint.
- ☐ **Public and private**
The cluster endpoint is accessible from outside of your VPC. Worker node traffic to the endpoint will stay within your VPC.
- ☐ **Private**
The cluster endpoint is only accessible through your VPC. Worker node traffic to the endpoint will stay within your VPC.

EKS Pod Identity (2)

Add-on name	IAM role	Service account
external-dns	Not set	external-dns
vpc-cni	Not set	aws-node

Screenshot: Cluster Creation in Progress

- Click **Create**.
- The EKS cluster enters the **Creating** state.

☒ Add-on(s) vpc-cni, eks-node-monitoring-agent, coredns, kube-proxy, external-dns, metrics-server, eks-pod-identity-agent successfully added to cluster mycluster.

Clusters (1) [Info](#)

Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created	Provider
mycluster	Creating	1.35	Standard support until March 27, 2027	Standard support	a few seconds ago	EKS

Screenshot: Launch EC2 Instance

- Go to the EC2 Dashboard.
- Click Launch Instance.
- Enter the Instance Name as eks-access.
- Select Amazon Linux AMI.
- Click Launch Instance.
- EC2 instance eks-access created successfully for communication with EKS worker nodes.

Instances (1/1) Info Last updated: less than a minute ago Connect Instance state Actions Launch instances

Saved filter sets: Choose filter set

Instance state = running Clear filters

☒	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
☒	eks-access	i-0be8034c67b94fa2e	Running	m7i-flex.large	Initializing	ap-south-1b	ec2-52-66-178-112.ap-...	52.66.178.112	-

Screenshot: EKS Cluster Active

- Wait for the cluster creation to complete.
- Verify that the cluster status changes from **Creating** to **Active**.

Clusters (1) Info Delete Create cluster

☒	Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created	Provider
☒	mycluster	Active	1.35 Upgrade now	Standard support until March 27, 2027	Standard support	9 minutes ago	EKS

Screenshot: Open Compute Tab

- Click on the **EKS Cluster**.
- Select the **Compute** tab.

mycluster Connect Delete cluster Upgrade version Monitor cluster

End of standard support for Kubernetes version 1.35 is March 27, 2027. Upgrade

Cluster info Info

Status Active

Cluster health 0

Kubernetes version 1.35 Info

Upgrade insights 0

Support period Standard support until March 27, 2027

Node health issues 0

Provider EKS

Capability issues 0

[Overview](#)
[Resources](#)
[Compute](#)
[Networking](#)
[Add-ons 1](#)
[Capabilities](#)
[Access](#)
[Observability](#)
[Update history & backups](#)
[Tags](#)

Screenshot: Add Node Group

- Click **Add Node Group**.

Node groups (0) Info Edit Delete Add

Node groups implement basic compute scaling through EC2 Auto Scaling groups.

Group name	Desired size	AMI release version	Launch template	Status
No node groups This cluster does not have any node groups. Nodes that are not part of an Amazon EKS managed node group are not shown in the AWS console.				

Add node group

Screenshot: Configure Node Group

- Enter the **Node Group Name**.

- Select the **Node IAM Role**.
- Click **Next** to continue.

Configure node group [Info](#)

A node group is a group of EC2 instances that supply compute capacity to your Amazon EKS cluster. You can add multiple node groups to your cluster.

Node group configuration

These properties cannot be changed after the node group is created.

Name

Assign a unique name for this node group.

The node group name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 63.

Node IAM role

To access other AWS services and resources your Nodes needs an IAM role. Choose a role with the right permissions for your Nodes.


[Create new role](#)

[View role details in IAM](#)

Screenshot: Configure Node Group Instance

- Select the **AMI Type**.
- Select **On-Demand** capacity.
- Select the **Available Instance Type**.
- Click **Next** to continue.

Set compute and scaling configuration

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type [Info](#)

Select the EKS-optimized Amazon Machine Image for nodes.

Capacity type

Select the capacity purchase option for this node group.

Instance types [Info](#)

Select instance types you prefer for this node group.

vCPU: 2 vCPUs Memory: 8 GiB Network: Up to 12.5 Gigabit Max ENI: 3 Max IPs: 30

Screenshot: Node Group Active

- Wait for the node group creation to complete.
- Verify that the **Node Group Status** changes to **Active**.

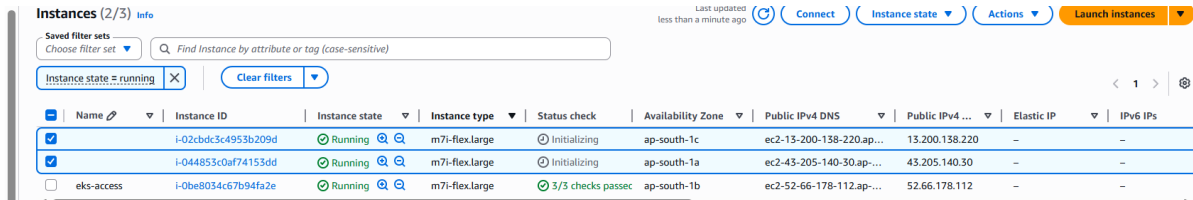
The screenshot shows the Amazon Elastic Kubernetes Service (EKS) console for a cluster named 'mycluster'. The cluster status is 'Active'. The 'Compute' tab is selected, displaying a table of nodes and a table of node groups.

Node name	Instance type	Compute	Managed by	Created	Status	CPU usage	Memory usage	Ephemeral storage usage
ip-172-31-31-244.ap-south-1.compute.internal	m7i-flex.large	Node group	mynodegrp	a minute ago	Ready	32%	17%	15%
ip-172-31-44-210.ap-south-1.compute.internal	m7i-flex.large	Node group	mynodegrp	a minute ago	Ready	12%	9%	15%

Group name	Desired size	AMI release version	Launch template	Status
mynodegrp	2	1.35.5-20260529	-	Active

Screenshot: Verify Worker Nodes in EC2

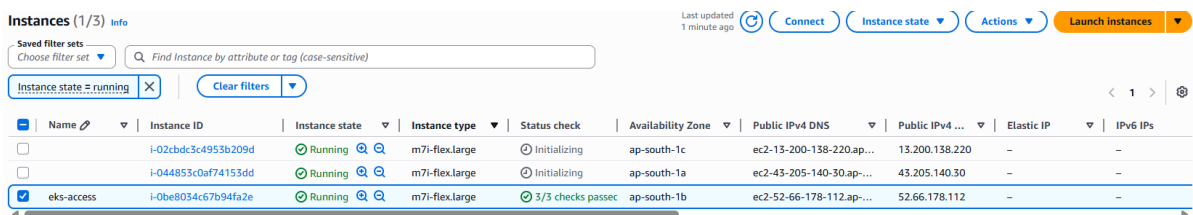
- Go to the **EC2 Dashboard**.
- Click **Instances**.
- Verify that **2 Worker Node instances** are available.



Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
	i-02cbdc3c4953b209d	Running	m7i-flex.large	Initializing	ap-south-1c	ec2-13-200-138-220.ap...	13.200.138.220	-	-
	i-044853c0af74153dd	Running	m7i-flex.large	Initializing	ap-south-1a	ec2-43-205-140-30.ap...	43.205.140.30	-	-
eks-access	i-0be8034c67b94fa2e	Running	m7i-flex.large	3/3 checks passed	ap-south-1b	ec2-52-66-178-112.ap...	52.66.178.112	-	-

Screenshot: Configure AWS CLI

- Connect to the **eks-access EC2 instance**.
- Run the command: `aws configure`
- Enter the **AWS Access Key ID**.
- Enter the **AWS Secret Access Key**.
- Enter the **Default Region**.
- Press **Enter** for the output format or set it if required.
- AWS CLI configured successfully.



Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
	i-02cbdc3c4953b209d	Running	m7i-flex.large	Initializing	ap-south-1c	ec2-13-200-138-220.ap...	13.200.138.220	-	-
	i-044853c0af74153dd	Running	m7i-flex.large	Initializing	ap-south-1a	ec2-43-205-140-30.ap...	43.205.140.30	-	-
eks-access	i-0be8034c67b94fa2e	Running	m7i-flex.large	3/3 checks passed	ap-south-1b	ec2-52-66-178-112.ap...	52.66.178.112	-	-

```
[ec2-user@ip-172-31-8-99 ~]$ aws configure
AWS Access Key ID [None]: AKIAUMRHU5GJGEMIPVMK
AWS Secret Access Key [None]: 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
Default region name [None]: Default output format [None]: [ec2-user@ip-172-31-8-99 ~]$
[ec2-user@ip-172-31-8-99 ~]$
```

Screenshot: Update kubeconfig

- Connect to the **eks-access EC2 instance**.
- Run the following command:

```
aws eks --region ap-south-1 update-kubeconfig --name mycluster
```

- The kubeconfig file is updated successfully.
- The EC2 instance is now configured to communicate with the EKS cluster.

```
[ec2-user@ip-172-31-8-99 ~]$ aws eks --region ap-south-1 update-kubeconfig --name mycluster
Added new context arn:aws:eks:ap-south-1:301804611986:cluster/mycluster to /home/ec2-user/.kube/config
[ec2-user@ip-172-31-8-99 ~]$
```

mycluster

End of standard support for Kubernetes version 1.35 is March 27, 2027. Upgrade

Cluster info Info

Status Active	Kubernetes version 1.35	Support period Standard support until March 27, 2027	Provider EKS
Cluster health 0	Upgrade insights 0	Node health issues 0	Capability issues 0

Overview Resources Compute Networking Add-ons 1 Capabilities Access Observability Update history & backups Tags

Screenshot: Create DaemonSet YAML File

- Run vi daemonset.yaml.
- Paste the DaemonSet configuration with the Nginx image.
- Press Esc.
- Type :wq!.
- Press Enter to save and exit the file.
- DaemonSet YAML file created successfully.

```
[ec2-user@ip-172-31-4-149 ~]$ vi daemonset.yaml
[ec2-user@ip-172-31-4-149 ~]$
```

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: my-daemonset
  labels:
    app: my-daemon-app
spec:
  selector:
    matchLabels:
      app: my-daemon-app
  template:
    metadata:
      labels:
        app: my-daemon-app
    spec:
      containers:
        - name: nginx-container
          image: nginx:latest
          ports:
            - containerPort: 80
```

Screenshot: Create DaemonSet

- Run kubectl apply -f daemonset.yaml.
- DaemonSet created successfully.

```
[ec2-user@ip-172-31-4-149 ~]$ ls
daemonset.yaml
[ec2-user@ip-172-31-4-149 ~]$ kubectl apply -f daemonset.yaml
daemonset.apps/my-daemonset created
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Verify DaemonSet Pods

- Run `kubectl get pods -o wide`.
- Press Enter.
- DaemonSet Pods are displayed successfully.
- One Pod is running on each Worker Node.

```
[ec2-user@ip-172-31-4-149 ~]$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
my-daemonset-94c7r   1/1     Running   0           47s
my-daemonset-kf79p   1/1     Running   0           47s
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Create Service YAML File

- Run `vi service.yaml`.
- Paste the Service configuration with type LoadBalancer.
- Press Esc.
- Type `:wq!`.
- Press Enter to save and exit the file.
- Service YAML file created successfully.

```
[ec2-user@ip-172-31-4-149 ~]$ vi service.yaml
[ec2-user@ip-172-31-4-149 ~]$
```

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  selector:
    app: my-daemon-app
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 80      # Service port
      targetPort: 80 # Pod container port
```

Screenshot: Create LoadBalancer Service

- Run `kubectl apply -f service.yaml`.
- LoadBalancer Service created successfully.

```
[ec2-user@ip-172-31-4-149 ~]$
[ec2-user@ip-172-31-4-149 ~]$ kubectl apply -f service.yaml
service/my-app-service created
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Verify Load Balancer

- Open EC2 Console.

- Click **Load Balancers**.
- Verify the Load Balancer is created successfully.

Load balancers (1/1) [What's new?](#)

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

☒	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones	Security groups
☒	aa7cf28b49dd84c2c82...	-	classic	-	-	vpc-00e925f7d875b3505	3 Availability Zones	sg-0f3476c05722

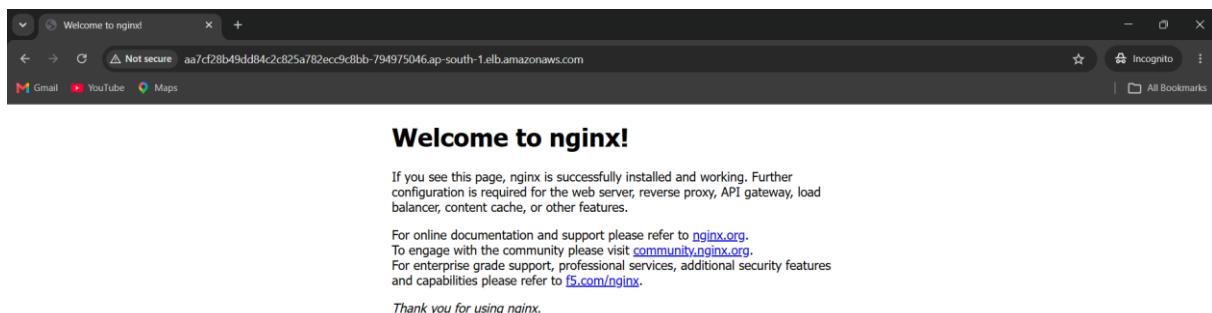
Screenshot: Copy DNS Name

- Select the created Load Balancer.
- Copy the **DNS Name**.
- DNS Name copied successfully.

		subnet-00cc3409b9fc886f5	ap-south-1c (aps1-az2)
DNS name Info aa7cf28b49dd84c2c825a782ecc9c8bb-794975046.ap-south-1.elb.amazonaws.com (A Record)			
ⓘ This Classic Load Balancer can be migrated to a next generation load balancer. Migration wizard uses your load balancer's current configurations to create a new load balancer. Learn more			
Launch migration wizard			

Screenshot: Access Nginx Application

- Open a new browser tab.
- Paste the Load Balancer DNS Name in the address bar.
- The Nginx welcome page is displayed successfully.



Screenshot: Edit Node Group Scaling Configuration

- Select the Node Group.
- Click **Edit**.
- Change **Desired size** from 2 to 3.
- Change **Minimum size** from 2 to 3.
- Change **Maximum size** from 2 to 3.
- Click **Save changes**.

Node groups (1) [Info](#)

Node groups implement basic compute scaling through EC2 Auto Scaling groups.

Q Filter node groups by property or value

< 1 >

Group name	Desired size	AMI release version	Launch template	Status
nodegrp	2	1.35.5-20260529	-	Active

Edit node group: nodegrp

Node group scaling configuration

Desired size

Set the desired number of nodes that the group should launch with initially.

3 nodes

Desired node size must be greater than or equal to 0

Minimum size

Set the minimum number of nodes that the group can scale in to.

3 nodes

Minimum node size must be greater than or equal to 0

Maximum size

Set the maximum number of nodes that the group can scale out to.

3 nodes

Maximum node size must be greater than or equal to 1 and cannot be lower than the minimum size

Screenshot: Verify Pods After Node Group Scaling

- Run `kubectl get pods`
- Three Pods are displayed successfully.
- A new Pod is created automatically on the newly added Worker Node.

```
[ec2-user@ip-172-31-4-149 ~]$  
[ec2-user@ip-172-31-4-149 ~]$ kubectl get pods  
NAME                                READY   STATUS    RESTARTS   AGE  
my-daemonset-94c7r                 1/1     Running   0           8m2s  
my-daemonset-kf79p                 1/1     Running   0           8m2s  
my-daemonset-s58lq                 1/1     Running   0           35s  
[ec2-user@ip-172-31-4-149 ~]$
```

STATEFULSET

Screenshot: Create StatefulSet Service YAML File

- Run `vi service-st.yaml`.
- Paste the Service configuration for the MySQL StatefulSet.
- Press Esc.
- Type `:wq!`.
- Press Enter to save and exit the file.
- StatefulSet Service YAML file created successfully.

```
[ec2-user@ip-172-31-4-149 ~]$  
[ec2-user@ip-172-31-4-149 ~]$ vi statefulset.yaml  
[ec2-user@ip-172-31-4-149 ~]$
```

```
apiVersion: apps/v1  
kind: StatefulSet  
metadata:  
  name: mysql-stateful  
spec:  
  serviceName: mysql-headless  
  replicas: 2  
  selector:  
    matchLabels:  
      app: mysql  
  
  template:  
    metadata:  
      labels:  
        app: mysql  
    spec:  
      containers:  
        - name: mysql  
          image: mysql:8.0  
  
          ports:  
            - containerPort: 3306  
  
          env:  
            - name: MYSQL_ROOT_PASSWORD  
              value: root123  
            - name: MYSQL_DATABASE  
              value: mydb  
  
          volumeMounts:  
            - name: mysql-storage  
              mountPath: /var/lib/mysql
```

```
volumeClaimTemplates:  
  - metadata:  
      name: mysql-storage  
    spec:  
      storageClassName: gp2  
      accessModes:  
        - ReadWriteOnce  
      resources:  
        requests:  
          storage: 2Gi  
-- INSERT --
```

Screenshot: Create StatefulSet

- Run `kubectl apply -f statefulset.yaml`.
- StatefulSet created successfully.

```
[ec2-user@ip-172-31-4-149 ~]$ kubectl apply -f statefulset.yaml  
statefulset.apps/mysql-stateful created  
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Verify MySQL Pods

- Run `kubectl get pods`.
- Two MySQL Pods are displayed successfully.
- Both MySQL Pods are in the **Running** state.

```
[ec2-user@ip-172-31-4-149 ~]$ kubectl get pods  
NAME                READY   STATUS    RESTARTS   AGE  
mysql-stateful-0     1/1     Running   0           45s  
mysql-stateful-1     1/1     Running   0           26s  
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Create StatefulSet Service YAML File

- Run vi service-st.yaml.
- Paste the Service configuration for the MySQL StatefulSet.
- Press Esc.
- Type :wq!.
- Press Enter to save and exit the file.
- StatefulSet Service YAML file created successfully.

```
[ec2-user@ip-172-31-4-149 ~]$ vi service-st.yaml
[ec2-user@ip-172-31-4-149 ~]$
```

```
apiVersion: v1
kind: Service
metadata:
  name: mysql-headless
spec:
  clusterIP: None
  selector:
    app: mysql
  ports:
    - port: 3306
      name: mysql
```

Screenshot: Create StatefulSet Service

- Run kubectl apply -f service-st.yaml.
- StatefulSet Service created successfully.

```
[ec2-user@ip-172-31-4-149 ~]$ kubectl apply -f service-st.yaml
service/mysql-headless created
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Verify Services

- Run kubectl get svc.
- Service details are displayed successfully.
- The StatefulSet Service is listed in the output.

```
service/mysql-headless created
[ec2-user@ip-172-31-4-149 ~]$ kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
kubernetes          ClusterIP   10.100.0.1    <none>        443/TCP    17m
mysql-headless      ClusterIP   None          <none>        3306/TCP   33s
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Connect to MySQL Pod

- Run kubectl exec -it mysql-stateful-0 -- mysql -u root -p.
- Enter the MySQL root password.

- Press Enter.
- MySQL login completed successfully.
- Exit

```
[ec2-user@ip-172-31-4-149 ~]$ kubectl exec -it mysql-stateful-0 -- mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
mysql>
mysql> exit
```

Screenshot: Verify Persistent Volume Claims

- Run `kubectl get pvc`.
- Persistent Volume Claim (PVC) details are displayed successfully.
- The PVCs created by the MySQL StatefulSet are listed in the output.

```
[ec2-user@ip-172-31-4-149 ~]$ kubectl get pvc
NAME                                STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
mysql-storage-mysql-stateful-0      Bound    pvc-20ef00a7-153a-40d8-a577-bf4be7234a36   2Gi        RWO            gp2            <unset>                 6m38s
mysql-storage-mysql-stateful-1      Bound    pvc-4c796f86-f515-4874-af2d-1bb0850445b0   2Gi        RWO            gp2            <unset>                 6m19s
```

Screenshot: Verify StatefulSet

- Run `kubectl get statefulset`.
- Press Enter.
- StatefulSet details are displayed successfully.
- The MySQL StatefulSet is listed in the output.

```
[ec2-user@ip-172-31-4-149 ~]$ kubectl get statefulset
NAME                READY   AGE
mysql-stateful      2/2     7m25s
[ec2-user@ip-172-31-4-149 ~]$
```